Top 100 SQL Interview Questions and Solutions

From fundamentals to window functions, classic puzzles, data modeling and Databricks/Spark SQL. Every question has a short explanation and a working query.

Section	Questions
Section 1: SQL Fundamentals	1 - 15
Section 2: Joins	16 - 30
Section 3: Aggregation and GROUP BY	31 - 40
Section 4: Subqueries and CTEs	41 - 52
Section 5: Window Functions	53 - 68
Section 6: Classic Interview Problems	69 - 85
Section 7: Performance, Modeling and Data Engineering	86 - 100

Practice schema used in the examples

Most queries use this small e-commerce / HR schema. Table and column names are illustrative.

```
employees   (emp_id, emp_name, dept_id, manager_id, salary, hire_date, gender)
departments (dept_id, dept_name)
customers   (customer_id, customer_name, email, city, signup_date, status)
orders      (order_id, customer_id, order_date, amount, status)
order_items (order_id, product_id, quantity, unit_price)
products    (product_id, product_name, category, price)
sales       (sale_id, region, category, amount, sale_date)
logins      (user_id, login_date)        events (user_id, event_name, event_time)
```

How to use this guide

- Default SQL is PostgreSQL-style. Where MySQL, SQL Server or Oracle differ (dates, LIMIT/TOP, strings), a variant is shown.

- In an interview, state assumptions first: table grain, handling of NULLs and ties, and which database you are on.

- Talk through the logical order of execution (FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY) before writing the query.

- After writing a solution, mention edge cases (duplicates, NULLs, ties, empty groups) and how you would check performance (EXPLAIN, indexes).

- Do not memorize: re-type each solution on your own with sample data until the pattern is natural.

Section 1: SQL Fundamentals

Q1. What are DDL, DML, DCL and TCL?

DDL defines structure (CREATE, ALTER, DROP, TRUNCATE). DML manipulates data (INSERT, UPDATE, DELETE, MERGE; SELECT is often called DQL). DCL controls access (GRANT, REVOKE). TCL controls transactions (COMMIT, ROLLBACK, SAVEPOINT).

Q2. What is the difference between DELETE, TRUNCATE and DROP?

DELETE removes rows (can use WHERE), is logged row by row, fires triggers and can be rolled back. TRUNCATE removes all rows quickly, has no WHERE, and usually resets identity counters. DROP removes the whole object: structure, data, indexes.

```
DELETE FROM orders WHERE status = 'CANCELLED';
TRUNCATE TABLE staging_orders;
DROP TABLE staging_orders;
```

Q3. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping and cannot use aggregates. HAVING filters groups after aggregation.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
WHERE hire_date >= '2020-01-01'      -- row filter
GROUP BY dept_id
HAVING AVG(salary) > 60000;          -- group filter
```

Q4. Explain primary key, unique key and foreign key.

A primary key uniquely identifies each row and cannot be NULL (one per table). A unique key enforces uniqueness but can allow NULL (a table can have many). A foreign key references a primary/unique key in another table to enforce referential integrity.

```
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE
);
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  FOREIGN KEY (customer_id) REFERENCES customers (customer_id)
);
```

Q5. How does SQL handle NULL? How do you work with it?

NULL means unknown. Any comparison with NULL (=, <>, >) returns UNKNOWN, so use IS NULL / IS NOT NULL. COALESCE returns the first non-NULL value, NULLIF returns NULL if two values are equal (useful to avoid divide-by-zero). Aggregates ignore NULLs except COUNT(*).

```
SELECT * FROM employees WHERE manager_id IS NULL;
SELECT emp_name, COALESCE(manager_id, 0) FROM employees;
SELECT revenue / NULLIF(units, 0) AS price_per_unit FROM sales_summary;
```

Q6. UNION vs UNION ALL?

UNION removes duplicates (extra sort/hash step). UNION ALL keeps every row and is faster. Both need the same number of columns with compatible types. Prefer UNION ALL unless you truly need de-duplication.

```
SELECT customer_id FROM orders_2024
UNION ALL
SELECT customer_id FROM orders_2025;
```

Q7. CHAR vs VARCHAR?

CHAR(n) is fixed length and padded with spaces; VARCHAR(n) is variable length and stores only what is needed. Use CHAR for truly fixed-size codes (e.g., country code 'IN'), VARCHAR for everything else.

Q8. What is the logical order of execution of a SQL query?

FROM / JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT (window functions evaluated here) -> DISTINCT -> ORDER BY -> LIMIT / TOP. This is why a column alias defined in SELECT cannot be used in WHERE, but can be used in ORDER BY.

Q9. How do wildcards work with LIKE?

% matches any number of characters (including zero); _ matches exactly one character.

```
SELECT * FROM customers WHERE customer_name LIKE 'A%';   -- starts with A
SELECT * FROM customers WHERE customer_name LIKE '_a%';  -- second letter is a
SELECT * FROM customers WHERE email LIKE '%@gmail.com';  -- ends with
```

Q10. Is BETWEEN inclusive? What is the safest way to filter a date range?

BETWEEN is inclusive on both ends. With dates/timestamps, a half-open range (>= start AND < next_start) avoids missing or double-counting boundary rows, and keeps the predicate index-friendly.

```
SELECT * FROM orders
WHERE order_date >= '2025-01-01'
  AND order_date <  '2025-02-01';     -- all of January
```

Q11. DISTINCT vs GROUP BY?

Both de-duplicate. GROUP BY additionally lets you aggregate. For plain de-duplication, optimizers usually generate the same plan.

```
SELECT DISTINCT city FROM customers;
SELECT city, COUNT(*) AS customers FROM customers GROUP BY city;
```

Q12. What constraints exist in SQL?

NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT (and in some engines EXCLUDE / INDEX-backed constraints). They enforce data integrity at the database level.

```
CREATE TABLE employees (
  emp_id    INT PRIMARY KEY,
  emp_name  VARCHAR(100) NOT NULL,
  salary    DECIMAL(10,2) CHECK (salary > 0),
  status    VARCHAR(10) DEFAULT 'ACTIVE'
);
```

Q13. How do you use CASE WHEN? Categorize employees by salary.

CASE is SQL's if/else expression. It works in SELECT, WHERE, ORDER BY, GROUP BY and inside aggregates.

```
SELECT emp_name, salary,
  CASE WHEN salary >= 100000 THEN 'High'
       WHEN salary >=  50000 THEN 'Medium'
       ELSE 'Low' END AS salary_band
FROM employees;
```

Q14. Fetch the top 5 highest-paid employees.

Syntax differs by database; ANSI standard is FETCH FIRST.

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 5;   -- PostgreSQL / MySQL
SELECT TOP 5 emp_name, salary FROM employees ORDER BY salary DESC;     -- SQL Server
SELECT emp_name, salary FROM employees
ORDER BY salary DESC FETCH FIRST 5 ROWS ONLY;                          -- ANSI / Oracle
```

Q15. COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column)?

COUNT(*) counts all rows. COUNT(col) counts non-NULL values. COUNT(DISTINCT col) counts distinct non-NULL values.

```
SELECT COUNT(*)                AS total_rows,
       COUNT(manager_id)       AS rows_with_manager,
       COUNT(DISTINCT dept_id) AS distinct_depts
FROM employees;
```

Section 2: Joins

Q16. What are the types of joins?

INNER (only matches), LEFT (all left rows + matches), RIGHT (all right rows + matches), FULL OUTER (all rows from both), CROSS (cartesian product), SELF (table joined with itself). Semi-join and anti-join are expressed with EXISTS / NOT EXISTS.

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON d.dept_id = e.dept_id;
```

Q17. INNER JOIN vs LEFT JOIN: what is the practical difference?

INNER JOIN drops rows with no match. LEFT JOIN keeps every row from the left table and fills right-side columns with NULL where nothing matches.

```
SELECT c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id;  -- customers with no orders still appear
```

Q18. Find customers who have never placed an order.

Classic anti-join. Use LEFT JOIN ... IS NULL or NOT EXISTS.

```
-- LEFT JOIN anti-join
SELECT c.*
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;

-- NOT EXISTS
SELECT c.*
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Q19. Show each employee with their manager's name (self join).

Join the table to itself using aliases; LEFT JOIN keeps the top-level employee who has no manager.

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM employees e
LEFT JOIN employees m ON m.emp_id = e.manager_id;
```

Q20. What is a CROSS JOIN and when is it useful?

It returns the cartesian product (rows = m x n). Useful for generating combinations, e.g., every store for every date to build a scaffold, then LEFT JOIN actual facts to expose missing days.

```
SELECT s.store_id, d.cal_date
FROM stores s
CROSS JOIN dim_date d;
```

Q21. What is a FULL OUTER JOIN? How to emulate it in MySQL?

It returns matched rows plus unmatched rows from both sides. MySQL has no FULL JOIN, so UNION a LEFT and a RIGHT join.

```
SELECT COALESCE(a.id, b.id) AS id, a.val AS a_val, b.val AS b_val
FROM a FULL OUTER JOIN b ON a.id = b.id;

-- MySQL emulation
SELECT * FROM a LEFT JOIN b ON a.id = b.id
UNION
SELECT * FROM a RIGHT JOIN b ON a.id = b.id;
```

Q22. Join vs subquery: when do you choose which?

Use a join when you need columns from both tables. Use EXISTS/IN subqueries when you only need an existence check and want to avoid row multiplication. Modern optimizers often rewrite one into the other, so favour clarity and correctness, then check the execution plan.

Q23. Find departments with no employees, and employees with no valid department.

Both are anti-joins in opposite directions.

```
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
WHERE e.emp_id IS NULL;

SELECT e.emp_name
FROM employees e
LEFT JOIN departments d ON d.dept_id = e.dept_id
WHERE d.dept_id IS NULL;
```

Q24. Why can NOT IN return no rows? How do you fix it?

If the subquery returns even one NULL, 'x NOT IN (..., NULL)' evaluates to UNKNOWN for every row, so nothing is returned. Use NOT EXISTS (NULL-safe) or filter NULLs out of the subquery.

```
-- risky
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);

-- safe
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);
```

Q25. What is a non-equi join? Give an example.

A join whose condition uses operators other than equality (BETWEEN, <, >). Typical for range lookups like salary grades or slowly-changing dimension validity ranges.

```
SELECT e.emp_name, g.grade
FROM employees e
JOIN salary_grades g ON e.salary BETWEEN g.min_sal AND g.max_sal;
```

Q26. Revenue by city and product category (joining 4 tables).

Chain joins from customers to orders to items to products, then aggregate at the grain you need.

```
SELECT c.city, p.category,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM customers c
JOIN orders o       ON o.customer_id = c.customer_id
JOIN order_items oi ON oi.order_id   = o.order_id
JOIN products p     ON p.product_id  = oi.product_id
GROUP BY c.city, p.category;
```

Q27. ON vs WHERE in a LEFT JOIN: what is the trap?

A WHERE filter on a right-table column eliminates the NULL-extended rows and silently turns the LEFT JOIN into an INNER JOIN. Put right-table conditions in ON to keep unmatched left rows.

```
-- keeps ALL customers, counts only completed orders
SELECT c.customer_id, COUNT(o.order_id) AS completed_orders
FROM customers c
LEFT JOIN orders o
       ON o.customer_id = c.customer_id AND o.status = 'COMPLETED'
GROUP BY c.customer_id;
```

Q28. IN vs EXISTS?

EXISTS stops at the first match and is NULL-safe; it shines when the inner table is large and correlated with an index. IN is fine for small, static lists. On modern optimizers performance is often identical, but NOT IN has the NULL trap (see earlier question).

```
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id = c.customer_id AND o.amount > 1000);
```

Q29. Why do SUMs get inflated after a join (fan-out)? How do you prevent it?

Joining a parent to a one-to-many child repeats parent rows once per child; summing a parent-level column then over-counts. Aggregate the child to the parent grain before joining, or aggregate at the correct level.

```
-- wrong: o.amount repeated once per order item
SELECT SUM(o.amount)
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id;

-- right: pre-aggregate items to one row per order
WITH items AS (
  SELECT order_id, SUM(quantity) AS qty FROM order_items GROUP BY order_id
)
SELECT SUM(o.amount) AS revenue, SUM(i.qty) AS units
FROM orders o JOIN items i ON i.order_id = o.order_id;
```

Q30. What are NATURAL JOIN and USING?

NATURAL JOIN auto-joins on all identically named columns, which is fragile (a new column can silently change results) and is discouraged. USING(col) is a concise, explicit alternative when both tables share the join column name.

```
SELECT emp_name, dept_name
FROM employees JOIN departments USING (dept_id);
```

Section 3: Aggregation and GROUP BY

Q31. Count orders per customer, including customers with zero orders.

LEFT JOIN from customers and COUNT a column from the orders side (not COUNT(*), which would count 1 for a customer with no orders).

```
SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS order_cnt
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name;
```

Q32. Departments with more than 5 employees, with headcount and average salary.

Group by department, use HAVING on the aggregate, sort by the metric.

```
SELECT d.dept_name, COUNT(*) AS headcount, ROUND(AVG(e.salary), 2) AS avg_salary
FROM employees e
JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_name
HAVING COUNT(*) > 5
ORDER BY avg_salary DESC;
```

Q33. Total revenue per month.

Truncate the date to month and group. Function names differ per database.

```
-- PostgreSQL
SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS revenue
FROM orders GROUP BY 1 ORDER BY 1;

-- MySQL
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(amount) AS revenue
FROM orders GROUP BY 1 ORDER BY 1;

-- SQL Server
SELECT FORMAT(order_date, 'yyyy-MM') AS month, SUM(amount) AS revenue
FROM orders GROUP BY FORMAT(order_date, 'yyyy-MM');
```

Q34. Customers with more than 3 orders in 2025.

Filter the year with WHERE (sargable range), group, then HAVING.

```
SELECT customer_id, COUNT(*) AS orders_2025
FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'
GROUP BY customer_id
HAVING COUNT(*) > 3;
```

Q35. How do you find duplicate records?

Group by the columns that define a duplicate and keep groups with COUNT(*) > 1.

```
SELECT email, COUNT(*) AS cnt
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Q36. How do you delete duplicates but keep one row?

Number rows within each duplicate group with ROW_NUMBER and delete rn > 1.

```
-- PostgreSQL / MySQL 8
DELETE FROM customers
WHERE customer_id IN (
  SELECT customer_id FROM (
    SELECT customer_id,
           ROW_NUMBER() OVER (PARTITION BY email ORDER BY customer_id) AS rn
    FROM customers) t
  WHERE rn > 1);

-- SQL Server (delete through the CTE)
WITH d AS (
  SELECT ROW_NUMBER() OVER (PARTITION BY email ORDER BY customer_id) AS rn
  FROM customers)
DELETE FROM d WHERE rn > 1;
```

Q37. What are ROLLUP, CUBE and GROUPING SETS?

They compute several aggregation levels in one pass. ROLLUP gives hierarchical subtotals plus a grand total, CUBE gives all combinations, GROUPING SETS lets you list exactly which groupings you want.

```
SELECT region, category, SUM(amount) AS sales
FROM sales
GROUP BY ROLLUP (region, category);       -- region+category, region, grand total

SELECT region, category, SUM(amount) AS sales
FROM sales
GROUP BY GROUPING SETS ((region, category), (region), ());
```

Q38. What is conditional aggregation?

Putting CASE inside an aggregate to compute several filtered metrics in one scan (also the standard way to pivot).

```
SELECT customer_id,
  SUM(CASE WHEN status = 'COMPLETED' THEN amount ELSE 0 END) AS completed_amt,
  SUM(CASE WHEN status = 'CANCELLED' THEN amount ELSE 0 END) AS cancelled_amt,
  COUNT(CASE WHEN status = 'RETURNED' THEN 1 END)            AS returns
FROM orders
GROUP BY customer_id;
```

Q39. Calculate each category's percentage of total revenue.

Combine an aggregate with a window aggregate over the whole result: SUM(SUM(x)) OVER ().

```
SELECT category,
       SUM(amount) AS revenue,
       ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS pct_of_total
FROM sales
GROUP BY category;
```

Q40. Monthly active customers and order count.

COUNT(DISTINCT ...) gives unique customers; COUNT(*) gives orders. Always state the grain.

```
SELECT DATE_TRUNC('month', order_date) AS month,
       COUNT(DISTINCT customer_id)     AS active_customers,
       COUNT(*)                        AS orders
FROM orders
GROUP BY 1
ORDER BY 1;
```

Section 4: Subqueries and CTEs

Q41. Find the second highest salary (3 ways).

Subquery with MAX, DENSE_RANK (handles ties and generalizes to Nth), or OFFSET.

```
-- 1. Nested MAX
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- 2. DENSE_RANK
SELECT DISTINCT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees) t
WHERE rnk = 2;

-- 3. OFFSET
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

Q42. Find the Nth highest salary.

Use DENSE_RANK so duplicate salaries count as one rank, then filter on N.

```
SELECT DISTINCT salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
      FROM employees) t
WHERE rnk = 3;      -- replace 3 with N
```

Q43. Employees earning more than the company average.

Use a scalar subquery for the average.

```
SELECT emp_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q44. Employees who earn more than their manager.

Self join and compare the two salaries.

```
SELECT e.emp_name, e.salary, m.emp_name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

Q45. Highest-paid employee in each department (correlated subquery).

A correlated subquery references the outer row. Returns all tied top earners.

```
SELECT *
FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees WHERE dept_id = e.dept_id);
```

Q46. CTE vs subquery vs temp table vs view?

CTE: named, readable, exists only for one statement, supports recursion. Subquery: inline, anonymous. Temp table: physically stored for the session, can be indexed and reused across statements, good for heavy intermediate results. View: saved query definition, no data stored, reusable and secure access layer.

```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id
)
SELECT e.emp_name, e.salary, d.avg_sal
FROM employees e
JOIN dept_avg d ON d.dept_id = e.dept_id
WHERE e.salary > d.avg_sal;
```

Q47. What is a recursive CTE? Print the full org hierarchy.

An anchor query plus a recursive member joined to the CTE itself, combined with UNION ALL until no new rows appear. Used for hierarchies, bill of materials, graph paths.

```
WITH RECURSIVE org AS (
  SELECT emp_id, emp_name, manager_id, 1 AS level
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.emp_name, e.manager_id, o.level + 1
  FROM employees e
  JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org ORDER BY level;
-- SQL Server: drop the RECURSIVE keyword
```

Q48. Generate a series of dates without a calendar table.

Use a recursive CTE, or a built-in generator where available.

```
WITH RECURSIVE dates AS (
  SELECT DATE '2025-01-01' AS d
  UNION ALL
  SELECT d + INTERVAL '1 day' FROM dates WHERE d < DATE '2025-01-31'
)
SELECT d FROM dates;

-- PostgreSQL shortcut
SELECT generate_series('2025-01-01'::date, '2025-01-31'::date, '1 day');
```

Q49. Customers who bought every product in the 'Electronics' category (relational division).

Count distinct matching products per customer and compare it with the total number of products in the category.

```
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id AND p.category = 'Electronics'
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) =
       (SELECT COUNT(*) FROM products WHERE category = 'Electronics');
```

Q50. Find products that have never been ordered.

Anti-join with NOT EXISTS.

```
SELECT p.product_id, p.product_name
FROM products p
WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id = p.product_id);
```

Q51. Which department has the highest average salary?

Aggregate, sort, take the top. Use RANK() instead of LIMIT 1 if ties must all be returned.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
ORDER BY avg_sal DESC
LIMIT 1;
```

Q52. What types of subqueries exist?

Scalar (returns one value), row/column (returns a row or list used with IN/ANY/ALL), table/derived (in FROM, needs an alias), and correlated (references the outer query and is logically re-evaluated per outer row).

```
SELECT emp_name,
       (SELECT COUNT(*) FROM employees x WHERE x.manager_id = e.emp_id) AS direct_reports
FROM employees e;       -- correlated scalar subquery
```

Section 5: Window Functions

Q53. What is a window function and how is it different from GROUP BY?

A window function computes a value across a set of related rows (the window) but keeps every input row; GROUP BY collapses rows. Syntax: func() OVER (PARTITION BY ... ORDER BY ... frame).

```
SELECT emp_name, dept_id, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;
```

Q54. ROW_NUMBER vs RANK vs DENSE_RANK?

ROW_NUMBER gives unique sequential numbers (1,2,3,4). RANK gives ties the same rank and leaves gaps (1,2,2,4). DENSE_RANK gives ties the same rank without gaps (1,2,2,3).

```
SELECT emp_name, salary,
  ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
  RANK()       OVER (ORDER BY salary DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk
FROM employees;
```

Q55. Top 3 highest-paid employees in each department.

Rank within each department and filter in an outer query (window functions cannot be used directly in WHERE).

```
SELECT * FROM (
  SELECT dept_id, emp_name, salary,
         DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
  FROM employees) t
WHERE rnk <= 3;
-- Snowflake / Databricks: use QUALIFY DENSE_RANK() OVER (...) <= 3 instead
```

Q56. Calculate a running total.

SUM over an ordered window with an explicit ROWS frame (deterministic when order values tie).

```
SELECT order_date, amount,
  SUM(amount) OVER (ORDER BY order_date, order_id
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM orders;
```

Q57. Calculate a 7-day moving average.

Use a frame of the current row and the 6 preceding rows (assumes one row per day; otherwise densify with a calendar first).

```
SELECT sale_date, daily_sales,
  AVG(daily_sales) OVER (ORDER BY sale_date
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS moving_avg_7d
FROM daily_sales;
```

Q58. Month-over-month revenue growth using LAG.

Aggregate to months, then compare each month to the previous one with LAG. Guard against divide-by-zero with NULLIF.

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS rev
  FROM orders GROUP BY 1
)
SELECT month, rev,
       LAG(rev) OVER (ORDER BY month) AS prev_rev,
       ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY month))
             / NULLIF(LAG(rev) OVER (ORDER BY month), 0), 2) AS mom_pct
FROM m;
```

Q59. Get the first (or latest) order for each customer.

ROW_NUMBER partitioned by customer, ordered ascending for first or descending for latest.

```
SELECT * FROM (
  SELECT o.*,
         ROW_NUMBER() OVER (PARTITION BY customer_id
                            ORDER BY order_date, order_id) AS rn
  FROM orders o) t
WHERE rn = 1;
```

Q60. Split employees into salary quartiles.

NTILE(n) distributes ordered rows into n roughly equal buckets.

```
SELECT emp_name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS quartile
FROM employees;
```

Q61. FIRST_VALUE / LAST_VALUE: what is the common gotcha?

The default frame ends at the current row, so LAST_VALUE just returns the current row's value. Specify ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING.

```
SELECT emp_name, dept_id, salary,
  FIRST_VALUE(emp_name) OVER (PARTITION BY dept_id ORDER BY salary DESC) AS top_earner,
  LAST_VALUE(emp_name)  OVER (PARTITION BY dept_id ORDER BY salary DESC
      ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)           AS lowest_earner
FROM employees;
```

Q62. Calculate the median salary.

Use PERCENTILE_CONT where supported; otherwise average the middle one or two rows.

```
-- PostgreSQL / Oracle / Snowflake
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees;

-- Portable
SELECT AVG(salary) AS median_salary FROM (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
         COUNT(*) OVER () AS cnt
  FROM employees) t
WHERE rn IN (FLOOR((cnt + 1) / 2.0), CEIL((cnt + 1) / 2.0));
```

Q63. Show each employee's difference from the department average and share of department payroll.

Windowed AVG and SUM partitioned by department.

```
SELECT emp_name, dept_id, salary,
  salary - AVG(salary) OVER (PARTITION BY dept_id)                      AS diff_from_avg,
  ROUND(100.0 * salary / SUM(salary) OVER (PARTITION BY dept_id), 2)    AS pct_of_dept_payroll
FROM employees;
```

Q64. ROWS vs RANGE in window frames?

ROWS counts physical rows. RANGE works on values: all peer rows with the same ORDER BY value are included together. The default frame when ORDER BY is present is RANGE UNBOUNDED PRECEDING, so tied rows get identical running totals; use ROWS for row-by-row accumulation.

```
SELECT order_date, amount,
  SUM(amount) OVER (ORDER BY order_date
                    RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS range_total,
  SUM(amount) OVER (ORDER BY order_date
                    ROWS  BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS rows_total
FROM orders;
```

Q65. Year-over-year growth.

Aggregate by year, LAG one row back, compute the percentage change.

```
WITH y AS (
  SELECT EXTRACT(YEAR FROM order_date) AS yr, SUM(amount) AS rev
  FROM orders GROUP BY 1
)
SELECT yr, rev,
       LAG(rev) OVER (ORDER BY yr) AS prev_year_rev,
       ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY yr))
             / NULLIF(LAG(rev) OVER (ORDER BY yr), 0), 2) AS yoy_pct
FROM y;
```

Q66. Find users with 3 or more consecutive login days (gaps and islands).

Subtract a ROW_NUMBER (in days) from the date; consecutive dates collapse to the same constant, which becomes the island key.

```
WITH d AS (
  SELECT DISTINCT user_id, login_date FROM logins
),
g AS (
  SELECT user_id, login_date,
         login_date - ROW_NUMBER() OVER (PARTITION BY user_id
                                         ORDER BY login_date)::int AS grp
  FROM d              -- login_date is DATE: PostgreSQL date minus int = date
)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end,
       COUNT(*) AS streak_days
FROM g
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
-- MySQL: DATE_SUB(login_date, INTERVAL rn DAY); SQL Server: DATEADD(day, -rn, login_date)
```

Q67. Sessionize events: a new session starts after 30 minutes of inactivity.

Flag rows whose gap from the previous event exceeds 30 minutes (or have no previous event), then take a running SUM of the flags as session_id.

```
WITH p AS (
  SELECT user_id, event_time,
         LAG(event_time) OVER (PARTITION BY user_id ORDER BY event_time) AS prev_time
  FROM events
),
f AS (
  SELECT user_id, event_time,
         CASE WHEN prev_time IS NULL
                OR event_time - prev_time > INTERVAL '30 minutes'
              THEN 1 ELSE 0 END AS new_session
  FROM p
)
SELECT user_id, event_time,
       SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_time) AS session_id
FROM f;
```

Q68. Days between a customer's consecutive orders.

LAG fetches the previous order date within each customer; subtract.

```
SELECT customer_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY customer_id
                                          ORDER BY order_date) AS days_since_prev
FROM orders;
-- SQL Server: DATEDIFF(day, LAG(order_date) OVER (...), order_date)
```

Section 6: Classic Interview Problems

Q69. Find numbers that appear at least 3 times consecutively (table logs: id, num).

Use LAG (or LEAD) twice and compare.

```
SELECT DISTINCT num
FROM (SELECT num,
             LAG(num, 1) OVER (ORDER BY id) AS p1,
             LAG(num, 2) OVER (ORDER BY id) AS p2
      FROM logs) t
WHERE num = p1 AND num = p2;
```

Q70. Customers who ordered in both January and February (retention).

INTERSECT of the two monthly customer sets (or two EXISTS / a HAVING COUNT(DISTINCT month) = 2).

```
SELECT customer_id FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2025-02-01'
INTERSECT
SELECT customer_id FROM orders
WHERE order_date >= '2025-02-01' AND order_date < '2025-03-01';
```

Q71. Find gaps (missing IDs) in a sequence.

LEAD the next id and report where the difference is greater than 1.

```
SELECT id + 1 AS gap_start, next_id - 1 AS gap_end
FROM (SELECT id, LEAD(id) OVER (ORDER BY id) AS next_id FROM t) x
WHERE next_id - id > 1;
```

Q72. Swap 'M' and 'F' values in a gender column with a single UPDATE.

CASE evaluates against the old row values within one statement, so no temp column is needed.

```
UPDATE employees
SET gender = CASE gender WHEN 'M' THEN 'F'
                         WHEN 'F' THEN 'M'
                         ELSE gender END;
```

Q73. Employees hired in the last 30 days.

Compare hire_date to today minus 30 days. Date syntax is database-specific.

```
SELECT * FROM employees WHERE hire_date >= CURRENT_DATE - INTERVAL '30 days';  -- PostgreSQL
SELECT * FROM employees WHERE hire_date >= CURDATE() - INTERVAL 30 DAY;        -- MySQL
SELECT * FROM employees WHERE hire_date >= DATEADD(day, -30, GETDATE());       -- SQL Server
```

Q74. Which customers generate the top 80% of revenue (Pareto)?

Aggregate revenue per customer, compute cumulative share ordered by revenue descending, and filter.

```
WITH c AS (
  SELECT customer_id, SUM(amount) AS rev FROM orders GROUP BY customer_id
),
r AS (
  SELECT customer_id, rev,
         SUM(rev) OVER (ORDER BY rev DESC) * 1.0 / SUM(rev) OVER () AS cum_pct
  FROM c
)
SELECT * FROM r WHERE cum_pct <= 0.80;
-- to include the customer who crosses 80%, test (cum_pct - rev / total) < 0.80
```

Q75. Pivot monthly revenue into columns per category.

Conditional aggregation works in every database; SQL Server and Oracle also have PIVOT.

```
SELECT category,
  SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 1 THEN amount END) AS jan,
  SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 2 THEN amount END) AS feb,
  SUM(CASE WHEN EXTRACT(MONTH FROM sale_date) = 3 THEN amount END) AS mar
FROM sales
GROUP BY category;
```

Q76. Unpivot columns into rows.

UNION ALL one SELECT per column; or use UNPIVOT (SQL Server / Oracle), CROSS APPLY VALUES, or stack() in Spark SQL.

```
SELECT category, 'jan' AS month, jan AS amount FROM monthly_sales
UNION ALL
SELECT category, 'feb', feb FROM monthly_sales
UNION ALL
SELECT category, 'mar', mar FROM monthly_sales;
```

Q77. Top-selling product (by units) in each category.

Aggregate, rank within the category (window over an aggregate), filter rank 1.

```
WITH s AS (
  SELECT p.category, p.product_name, SUM(oi.quantity) AS units,
         RANK() OVER (PARTITION BY p.category
                      ORDER BY SUM(oi.quantity) DESC) AS rnk
  FROM order_items oi
  JOIN products p ON p.product_id = oi.product_id
  GROUP BY p.category, p.product_name
)
SELECT category, product_name, units FROM s WHERE rnk = 1;
```

Q78. Churned customers: bought in 2024 but not in 2025.

Set difference with EXCEPT (MINUS in Oracle), or an anti-join.

```
SELECT DISTINCT customer_id FROM orders
WHERE order_date >= '2024-01-01' AND order_date < '2025-01-01'
EXCEPT
SELECT DISTINCT customer_id FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
```

Q79. New vs returning customers per month.

Find each customer's first-order month; a customer is new in that month and returning afterwards.

```
WITH f AS (
  SELECT customer_id, DATE_TRUNC('month', MIN(order_date)) AS first_month
  FROM orders GROUP BY customer_id
)
SELECT DATE_TRUNC('month', o.order_date) AS month,
  COUNT(DISTINCT CASE WHEN DATE_TRUNC('month', o.order_date) = f.first_month
                      THEN o.customer_id END) AS new_customers,
  COUNT(DISTINCT CASE WHEN DATE_TRUNC('month', o.order_date) > f.first_month
                      THEN o.customer_id END) AS returning_customers
FROM orders o
JOIN f ON f.customer_id = o.customer_id
GROUP BY 1
ORDER BY 1;
```

Q80. Find employees who share the same salary with another employee.

Self-EXISTS with a different employee id (or GROUP BY salary HAVING COUNT(*) > 1 and join back).

```
SELECT e1.emp_name, e1.salary
FROM employees e1
WHERE EXISTS (SELECT 1 FROM employees e2
              WHERE e2.salary = e1.salary AND e2.emp_id <> e1.emp_id);
```

Q81. How do you compare two tables (source vs target reconciliation)?

EXCEPT in both directions finds missing/different rows; compare counts and sums as a quick sanity check.

```
SELECT * FROM source_tbl EXCEPT SELECT * FROM target_tbl;   -- missing or changed in target
SELECT * FROM target_tbl EXCEPT SELECT * FROM source_tbl;   -- extra or changed in target

SELECT (SELECT COUNT(*)    FROM source_tbl)  AS src_cnt,
       (SELECT COUNT(*)    FROM target_tbl)  AS tgt_cnt,
       (SELECT SUM(amount) FROM source_tbl)  AS src_amt,
       (SELECT SUM(amount) FROM target_tbl)  AS tgt_amt;
```

Q82. Common string functions: extract the email domain, proper-case a name, trim.

SUBSTRING / POSITION, LEFT, UPPER, LOWER, TRIM, LENGTH, CONCAT or ||, REPLACE, SPLIT_PART.

```
SELECT email,
  SUBSTRING(email FROM POSITION('@' IN email) + 1)                    AS domain,
  SPLIT_PART(email, '@', 2)                                           AS domain_alt,
  UPPER(LEFT(customer_name, 1)) || LOWER(SUBSTRING(customer_name FROM 2)) AS proper_name,
  LENGTH(TRIM(customer_name))                                         AS name_len
FROM customers;
-- MySQL: SUBSTRING_INDEX(email, '@', -1)   SQL Server: SUBSTRING(email, CHARINDEX('@', email) + 1, 100)
```

Q83. Common date functions: first/last day of month, tenure, date difference.

Know your database's equivalents: DATE_TRUNC, EOMONTH, DATEDIFF, AGE, INTERVAL arithmetic.

```
-- PostgreSQL
SELECT DATE_TRUNC('month', CURRENT_DATE)::date AS first_day,
       (DATE_TRUNC('month', CURRENT_DATE) + INTERVAL '1 month' - INTERVAL '1 day')::date AS last_day,
       CURRENT_DATE - hire_date               AS tenure_days,
       AGE(CURRENT_DATE, hire_date)           AS tenure
FROM employees;
-- SQL Server: DATEDIFF(day, hire_date, GETDATE()), EOMONTH(GETDATE())
```

Q84. Build a monthly cohort retention table.

Assign each customer to the month of their first order (cohort), then count distinct active customers by months since the cohort month.

```
WITH c AS (
  SELECT customer_id, DATE_TRUNC('month', MIN(order_date)) AS cohort_month
  FROM orders GROUP BY customer_id
),
a AS (
  SELECT DISTINCT customer_id, DATE_TRUNC('month', order_date) AS active_month
  FROM orders
)
SELECT c.cohort_month,
       (EXTRACT(YEAR FROM a.active_month) * 12 + EXTRACT(MONTH FROM a.active_month))
     - (EXTRACT(YEAR FROM c.cohort_month) * 12 + EXTRACT(MONTH FROM c.cohort_month)) AS month_no,
       COUNT(DISTINCT a.customer_id) AS active_customers
FROM c JOIN a ON a.customer_id = c.customer_id
GROUP BY 1, 2
ORDER BY 1, 2;
```

Q85. Compute a simple funnel: view -> cart -> purchase (table events: user_id, event_name).

Conditional COUNT(DISTINCT) per stage. This does not enforce ordering; for a strict funnel, join stages on user and timestamp.

```
SELECT COUNT(DISTINCT CASE WHEN event_name = 'view'     THEN user_id END) AS viewed,
       COUNT(DISTINCT CASE WHEN event_name = 'cart'     THEN user_id END) AS carted,
       COUNT(DISTINCT CASE WHEN event_name = 'purchase' THEN user_id END) AS purchased
FROM events;
```

Section 7: Performance, Modeling and Data Engineering

Q86. What is an index? Clustered vs non-clustered?

An index is a sorted structure (usually a B-tree) that speeds up lookups at the cost of extra storage and slower writes. A clustered index defines the physical order of the table rows (only one per table; the primary key in SQL Server / InnoDB). A non-clustered index is a separate structure with pointers to rows (many allowed). Also know composite, covering, unique, partial/filtered, bitmap and hash indexes.

```
CREATE INDEX idx_orders_cust_date ON orders (customer_id, order_date);
CREATE UNIQUE INDEX idx_cust_email ON customers (email);
```

Q87. When will the optimizer NOT use an index? (sargability)

Functions or arithmetic on the indexed column, leading wildcards, implicit type conversions, OR across different columns, low-selectivity columns, and not matching the leftmost prefix of a composite index. Rewrite predicates so the bare column is compared to a constant or range.

```
-- bad: function on the column
WHERE YEAR(order_date) = 2025
-- good: range on the bare column
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01'

-- bad: leading wildcard
WHERE customer_name LIKE '%son'
```

Q88. How do you read an execution plan (EXPLAIN)?

Check scan types (sequential vs index), join algorithms (nested loop, hash, merge), estimated vs actual row counts (large gaps mean stale statistics), expensive sorts or spills to disk, and the most costly nodes. Fix with indexes, rewritten predicates, updated stats, or less data scanned.

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 42;
```

Q89. Explain normalization (1NF, 2NF, 3NF). When do you denormalize?

1NF: atomic values, no repeating groups. 2NF: 1NF plus no partial dependency on part of a composite key. 3NF: 2NF plus no transitive dependency (non-key columns depend only on the key). BCNF is a stricter 3NF. Normalize OLTP systems to cut redundancy and anomalies; denormalize analytics models to reduce joins and speed up reads, accepting redundancy.

Q90. Star schema vs snowflake schema; fact vs dimension tables?

Fact tables hold measurable events (sales_amount, quantity) with foreign keys to dimensions, at a declared grain. Dimension tables hold descriptive context (customer, product, date). A star schema keeps dimensions denormalized and one join away from the fact: simpler and faster. A snowflake normalizes dimensions into sub-tables: less redundancy, more joins.

```
SELECT d.year, p.category, SUM(f.sales_amount) AS revenue
FROM fact_sales f
JOIN dim_date d    ON d.date_key    = f.date_key
JOIN dim_product p ON p.product_key = f.product_key
GROUP BY d.year, p.category;
```

Q91. Explain Slowly Changing Dimensions (Type 1, 2, 3). How is Type 2 implemented?

Type 1 overwrites (no history). Type 2 adds a new row per change with start_date, end_date and is_current (full history). Type 3 adds a column holding the previous value (limited history). Type 2 = expire the old row, insert the new version.

```
-- 1) expire current rows whose attributes changed (PostgreSQL syntax)
UPDATE dim_customer d
SET end_date = CURRENT_DATE, is_current = FALSE
FROM stg_customer s
WHERE d.customer_id = s.customer_id AND d.is_current
  AND d.city IS DISTINCT FROM s.city;

-- 2) insert new versions and brand-new customers
INSERT INTO dim_customer (customer_id, city, start_date, end_date, is_current)
SELECT s.customer_id, s.city, CURRENT_DATE, NULL, TRUE
FROM stg_customer s
LEFT JOIN dim_customer d ON d.customer_id = s.customer_id AND d.is_current
WHERE d.customer_id IS NULL;
```

Q92. What are ACID properties and transaction isolation levels?

Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent transactions do not interfere), Durability (committed data survives crashes). Isolation levels: Read Uncommitted (dirty reads possible), Read Committed (no dirty reads), Repeatable Read (no non-repeatable reads), Serializable (no anomalies, lowest concurrency). Phantom-read behaviour at Repeatable Read varies by engine.

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE acc_id = 2;
COMMIT;      -- ROLLBACK; if anything fails
```

Q93. What is a deadlock and how do you prevent it?

Two transactions each hold a lock the other needs, so neither can proceed; the database aborts one as the victim. Prevent by accessing objects in a consistent order, keeping transactions short, indexing to reduce lock scope, using appropriate isolation, and adding retry logic in the application.

Q94. View vs materialized view?

A view stores only the query definition and runs it each time. A materialized view physically stores the result, so reads are fast, but it must be refreshed and can be stale.

```
CREATE VIEW v_active_customers AS
SELECT * FROM customers WHERE status = 'ACTIVE';

CREATE MATERIALIZED VIEW mv_daily_sales AS
SELECT order_date, SUM(amount) AS revenue FROM orders GROUP BY order_date;

REFRESH MATERIALIZED VIEW mv_daily_sales;
```

Q95. Stored procedure vs function vs trigger?

A function returns a value and can be used inside queries (usually no side effects or transaction control). A stored procedure is invoked with CALL/EXEC, can modify data, return multiple results and control transactions. A trigger runs automatically on INSERT/UPDATE/DELETE; powerful for auditing but hides logic and can hurt performance.

```
CREATE FUNCTION gst(amount NUMERIC) RETURNS NUMERIC
AS $$ SELECT amount * 0.18 $$ LANGUAGE SQL;     -- PostgreSQL

SELECT order_id, amount, gst(amount) FROM orders;
```

Q96. What is MERGE / UPSERT and why is it used in incremental loads?

It inserts new rows and updates existing ones in a single atomic statement, which is the backbone of incremental/CDC loads.

```
MERGE INTO dim_customer t
USING stg_customer s ON t.customer_id = s.customer_id
WHEN MATCHED THEN
  UPDATE SET t.city = s.city, t.email = s.email
WHEN NOT MATCHED THEN
  INSERT (customer_id, city, email) VALUES (s.customer_id, s.city, s.email);

-- PostgreSQL: INSERT ... ON CONFLICT (customer_id) DO UPDATE SET city = EXCLUDED.city;
-- MySQL:      INSERT ... ON DUPLICATE KEY UPDATE city = VALUES(city);
```

Q97. How do you optimize a slow SQL query? What is partitioning?

Checklist: read the execution plan; select only needed columns (no SELECT *); filter early with sargable predicates; add or fix indexes; pre-aggregate before joining; replace correlated subqueries with joins/windows; use EXISTS for existence checks; refresh statistics; avoid DISTINCT as a band-aid for bad joins. Partitioning splits a large table by range/list/hash so queries scan only the relevant partitions (partition pruning) and maintenance is easier.

```
CREATE TABLE sales_p (sale_id BIGINT, sale_date DATE, amount NUMERIC)
PARTITION BY RANGE (sale_date);

CREATE TABLE sales_2025 PARTITION OF sales_p
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Q98. OLTP vs OLAP; row store vs column store?

OLTP: many small, concurrent transactional reads/writes, normalized schemas, row-oriented storage (e.g., PostgreSQL, MySQL). OLAP: large analytical scans and aggregations over history, denormalized/star schemas, column-oriented storage (e.g., Snowflake, Redshift, BigQuery, Parquet/Delta) that reads only needed columns and compresses well.

Q99. How do you validate data quality using SQL?

Typical checks: completeness (NULLs), uniqueness (duplicates on business keys), referential integrity (orphans), validity (ranges, formats, allowed values), freshness (latest timestamp), and reconciliation (row counts and totals between layers). These are the building blocks of data-testing frameworks.

```
-- completeness
SELECT COUNT(*) FROM orders WHERE customer_id IS NULL OR order_date IS NULL;

-- uniqueness
SELECT order_id, COUNT(*) FROM orders GROUP BY order_id HAVING COUNT(*) > 1;

-- referential integrity (orphans)
SELECT o.order_id FROM orders o
LEFT JOIN customers c ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL;

-- validity
SELECT * FROM orders WHERE amount < 0 OR order_date > CURRENT_DATE;

-- freshness
SELECT MAX(order_date) AS latest_order FROM orders;

-- reconciliation (raw vs curated)
SELECT (SELECT COUNT(*) FROM raw.orders) AS raw_cnt,
       (SELECT COUNT(*) FROM curated.orders) AS curated_cnt;
```

Q100. Spark SQL / Databricks (Delta Lake) essentials: MERGE, time travel, QUALIFY, broadcast hint, OPTIMIZE.

Delta Lake adds ACID transactions to the lakehouse. MERGE handles upserts, time travel reads older versions, QUALIFY filters on window functions, a BROADCAST hint sends a small table to every executor to avoid shuffles, and OPTIMIZE ... ZORDER compacts small files and co-locates data for faster filtering.

```
MERGE INTO silver.customers t
USING bronze.customer_updates s ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

SELECT * FROM silver.customers VERSION AS OF 5;                 -- time travel
SELECT * FROM silver.customers TIMESTAMP AS OF '2025-06-01';

SELECT /*+ BROADCAST(d) */ f.*, d.region                        -- broadcast join
FROM fact_sales f JOIN dim_store d ON d.store_id = f.store_id;

SELECT * FROM silver.orders                                     -- latest row per key
QUALIFY ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY updated_at DESC) = 1;

OPTIMIZE silver.orders ZORDER BY (customer_id);
```